

REPORTE FASE 1 y 2



García Torres Daniela Catalina
Espinoza Robles Fátima Gabriela
Recamier Acevedo José de Jesús
Lizaola Hernández Aylin Danae
Lomelí Mejía María del Sagrario
Ferrer Saucedo Valeria Lizeth

ARQUITECTURA DE COMPUTADORAS

1. Elementos y características generales del procesador MIPS de 32 bits

La arquitectura MIPS (Microprocessor without Interlocked Pipeline Stages) es uno de los ejemplos más representativos de la filosofía de diseño RISC (Reduced Instruction Set Computer). Su diseño se centra en la simplicidad, la eficiencia y el alto rendimiento mediante la ejecución rápida de instrucciones sencillas.

Las características principales de un procesador MIPS de 32 bits incluyen:

- **Arquitectura de 32 bits:** Los registros, el bus de datos y las direcciones de memoria tienen un tamaño de 32 bits (1 palabra).
- **Formato de instrucción fijo:** Todas las instrucciones tienen una longitud estricta de 32 bits. Esto simplifica enormemente el proceso de decodificación en la Unidad de Control.
- **Arquitectura Load/Store:** Las operaciones aritméticas y lógicas solo se realizan entre registros. El acceso a la memoria principal está estrictamente limitado a las instrucciones de carga (*Load*) y almacenamiento (*Store*).
- **Banco de Registros:** Cuenta con 32 registros de propósito general, lo que permite mantener las variables de uso frecuente dentro del procesador para evitar accesos lentos a la memoria.

2. Set de instrucciones MIPS y definición del ISA para el diseño

El conjunto de instrucciones (ISA) de MIPS se divide en tres formatos principales, dependiendo de cómo se distribuyen los 32 bits de la instrucción:

1. **Tipo R (Registro):** Utilizan tres registros (dos de origen y uno de destino).
2. **Tipo I (Inmediato):** Utilizan dos registros y un valor constante (inmediato) de 16 bits.
3. **Tipo J (Salto):** Utilizan un código de operación y una dirección de salto de 26 bits.

Set de instrucciones específico del proyecto: Para este diseño, se ha definido un ISA reducido basado en las especificaciones del proyecto. Las instrucciones que el procesador será capaz de ejecutar son:

- **Aritméticas y Lógicas (Tipo R):** Add, Sub, Or, And, slt, sltu, nor, xor, sll, srl, nop.
- **Aritméticas y Lógicas (Tipo I):** Addi, Ori, Andi, Slti, Xori, Sltui, Lui.
- **Acceso a Memoria (Tipo I):** Lw, Sw.
- **Control de Flujo (Tipo I y J):** beq, bne, bgtz (Tipo I), j y jal (Tipo J).

3. Desarrollo de los elementos implementados (Fase 1)

En esta primera fase del proyecto, el objetivo principal fue la implementación del *single datapath* (ruta de datos de un solo ciclo) de MIPS de 32 bits, enfocado exclusivamente en la ejecución de instrucciones de **Tipo R**.

Para lograr esto, se desarrollaron e interconectaron los siguientes módulos de hardware (basados en la Figura 1 y 2 del documento) :

- **Program Counter (PC):** Registro que almacena la dirección de la instrucción actual.
- **Memoria de Instrucciones:** Módulo de solo lectura donde se precargan las instrucciones en código máquina.
- **Banco de Registros (Registers):** Memoria interna que contiene los 32 registros. Para las instrucciones Tipo R, lee dos registros fuentes (**Read register 1** y **Read register 2**) y escribe en un registro destino (**Write register**).
- **ALU (Arithmetic Logic Unit):** Encargada de realizar las operaciones matemáticas (suma, resta) y lógicas (AND, OR) requeridas por la instrucción.
- **Unidades de Control (Control y ALU Control):** La unidad de control principal decodifica los 6 bits más significativos (Opcode). Para las instrucciones Tipo R, delega la decisión final al *ALU Control*, el cual lee los últimos 6 bits de la instrucción (campo *funct*) para determinar la operación exacta que la ALU debe realizar.

El flujo de ejecución de una instrucción Tipo R implementado es el siguiente: El PC indica la dirección a la Memoria de Instrucciones; la instrucción es extraída; los campos de los registros fuente pasan al Banco de Registros; los datos leídos pasan a la ALU, la cual es configurada por el ALU Control; finalmente, el resultado de la ALU se retroalimenta al Banco de Registros para ser guardado.

4. Opciones de algoritmos en ensamblador posibles con este ISA

Respondiendo a la interrogante sobre qué algoritmos se pueden implementar con el conjunto de instrucciones definido en las Tablas 1 y 2, las opciones son bastante amplias. Dado que el ISA incluye operaciones aritméticas completas, acceso a memoria y ramificaciones condicionales (saltos), este procesador es *Turing completo* en la práctica (limitado solo por el tamaño de la memoria).

Algunas opciones de algoritmos viables que podrían proponer para su proyecto son:

1. Cálculo de la Serie de Fibonacci:

- **Por qué es posible:** Requiere sumas (**Add**), inicialización de variables (**Addi**), y un ciclo para iterar n veces usando ramificaciones (**bne** o **bgtz**).

2. Factorial de un número (Iterativo):

- **Por qué es posible:** Se requiere multiplicar el resultado acumulado (**Mul**), restar 1 al contador en cada ciclo (**Subi**) y verificar si el contador ha llegado a cero (**bgtz**).

3. Búsqueda en un Arreglo (Búsqueda Lineal):

- **Por qué es posible:** Implica iterar sobre datos almacenados en memoria. Se usaría **Lw** para traer los datos del arreglo, **beq** o **bne** para comparar si se encontró el número buscado, y **Addi** para avanzar el puntero de memoria.

4. Ordenamiento Burbuja (Bubble Sort):

- **Por qué es posible:** Es uno de los algoritmos más completos para probar un procesador. Requiere manipular un arreglo usando **Lw** y **Sw**, comparar elementos adyacentes usando Set Less Than (**slt**), y realizar ciclos anidados manejados por **bne** y variables temporales.

5. Multiplicación de Matrices o Vectores:

- **Por qué es posible:** Usando **Lw**, operaciones matemáticas acumulativas (**Mul**, **Add**) y ciclos de control.

5. Desarrollo e implementación de la ruta de datos para instrucciones Tipo I

Durante la Fase 2, el diseño del *datapath* (ruta de datos) se expandió significativamente para soportar las instrucciones de Tipo I. A diferencia de las instrucciones Tipo R, que operan exclusivamente con registros, las instrucciones Tipo I incluyen un valor constante de 16 bits (inmediato) dentro de la misma instrucción. Este formato permite realizar operaciones aritméticas con constantes (como **Addi**), transferir datos entre los registros y la memoria principal (**Lw**, **Sw**), y evaluar condiciones para saltos condicionales (**beq**, **bne**).

Para integrar estas nuevas capacidades sin perder la funcionalidad de la Fase 1, fue necesario desarrollar nuevos módulos e incorporar multiplexores estratégicos que permitan a la Unidad de Control dirigir el flujo de datos según la instrucción en curso.

Nuevos elementos incorporados al Datapath:

- **Módulo de Extensión de Signo (Sign Extend):** Las operaciones de la ALU y las direcciones de memoria requieren datos de 32 bits, pero el valor inmediato en las instrucciones Tipo I es de solo 16 bits (bits 15 al 0). Este módulo toma esos 16 bits y los convierte en un valor de 32 bits replicando el bit más significativo (el bit de signo), preservando así el valor matemático correcto (positivo o negativo) antes de que ingrese a la ALU.
- **Memoria de Datos (Data Memory):** A diferencia de la Memoria de Instrucciones, la Memoria de Datos es de lectura y escritura. Se habilitó para soportar las instrucciones *Load Word (Lw)* y *Store Word (Sw)*. Cuenta con entradas para la dirección (calculada por la ALU) y el dato a escribir, así como señales de control (*MemRead* y *MemWrite*) dictadas por la Unidad de Control.
- **Sumador de Dirección de Salto (Branch Adder):** Para las instrucciones de salto condicional (como *beq*), la dirección de destino no está en un registro, sino que se calcula sumando el PC actual ($PC + 4$) con el valor inmediato extendido en signo y desplazado dos bits a la izquierda (Shift Left 2). Este sumador dedicado trabaja en paralelo con la ALU principal.

Modificaciones y Multiplexores de Enrutamiento:

Para que el mismo circuito físico pueda ejecutar tanto instrucciones Tipo R como Tipo I, se implementaron multiplexores clave controlados por la Unidad de Control:

- **Multiplexor ALUSrc (ALU Source):** Ubicado justo antes de la segunda entrada de la ALU. Decide si el segundo operando proviene del Banco de Registros (para instrucciones Tipo R como *Add*) o del módulo de Extensión de Signo (para instrucciones Tipo I como *Addi* o el cálculo de direcciones en *Lw/Sw*).
- **Multiplexor MemToReg (Memory to Register):** Ubicado en la etapa final de escritura del Banco de Registros. Selecciona qué dato se guardará en el registro destino: el resultado matemático proveniente de la ALU (para operaciones aritméticas) o el dato extraído de la Memoria de Datos (para la instrucción *Lw*).

- **Multiplexor RegDst (Register Destination):** Determina qué campo de la instrucción indica el registro de destino. En las instrucciones Tipo R, el destino es el campo **rd** (bits 15-11). Sin embargo, en las instrucciones Tipo I (como **Lw** o **Addi**), el destino es el campo **rt** (bits 20-16). Este multiplexor asegura que el dato se escriba en el registro correcto.
- **Multiplexor PCSrc (PC Source) y Puerta AND:** Controla el flujo del programa. Decide si el Program Counter (PC) avanza a la siguiente instrucción secuencial ($PC + 4$) o si toma la dirección calculada por el *Branch Adder*. Para que ocurra el salto, deben cumplirse dos condiciones evaluadas por una compuerta AND: que la Unidad de Control emita la señal **Branch** (es una instrucción de salto) y que la ALU emita la bandera **Zero** (la condición del salto se cumple).

Expansión de la Unidad de Control: Finalmente, el módulo de la Unidad de Control fue reescrito para decodificar los *Opcodes* distintos a cero. Ahora es capaz de reconocer instrucciones como **Addi**, **Lw** y **Sw**, activando las combinaciones correctas de las nuevas señales de control (**ALUSrc**, **MemToReg**, **RegDst**, **MemRead**, **MemWrite** y **Branch**) para sincronizar todo el *datapath*.

Conclusiones

En esta fase del proyecto se amplió el diseño del procesador para soportar un conjunto más amplio de instrucciones, incluyendo instrucciones tipo I como **addi**, **lw**, **sw**, **beq**, entre otras, así como instrucciones de salto. Para lograrlo fue necesario modificar la unidad de control, agregando nuevas señales de salida y actualizando el control de la ALU para manejar las operaciones correspondientes.

Se dio un primer paso hacia una arquitectura pipeline mediante la implementación de buffers de registro entre etapas (IF/ID, ID/EX, EX/MEM, MEM/WB), lo cual permite segmentar la ejecución de instrucciones para mejorar el rendimiento del procesador.

Durante las pruebas se verificó que las instrucciones aritméticas y de memoria funcionan correctamente, obteniendo los resultados esperados en el banco de registros y en la memoria de datos. En general, esta práctica nos permitió comprender mejor la arquitectura del procesador y la forma en que los datos y las señales de control se desplazan a través de cada etapa de ejecución.

Finalmente, durante el desarrollo de esta práctica nos dimos cuenta del reto que implica implementar un pipeline funcional, especialmente en el manejo de instrucciones y del flujo de datos, aspectos importantes para lograr una ejecución eficiente dentro del procesador.